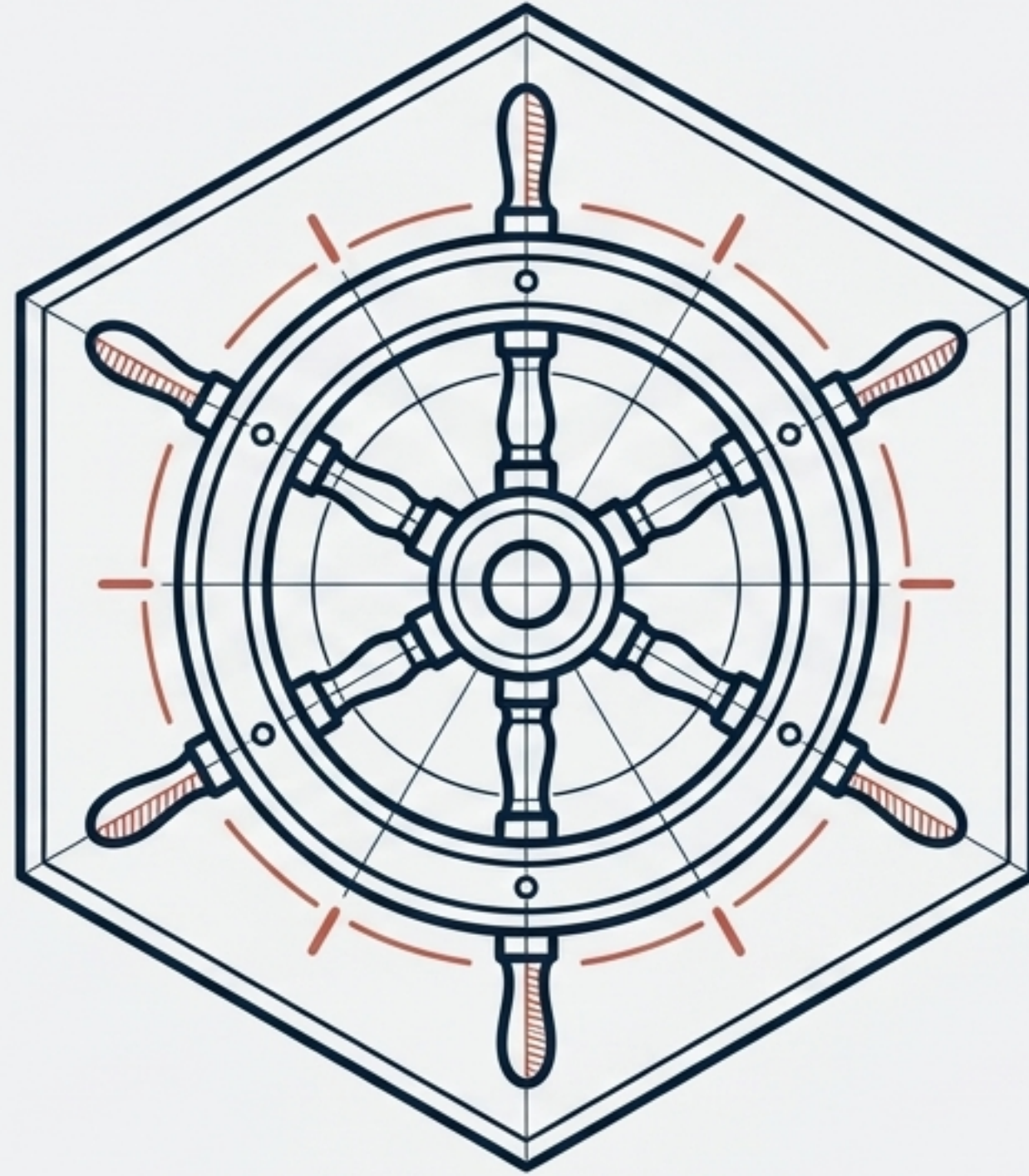




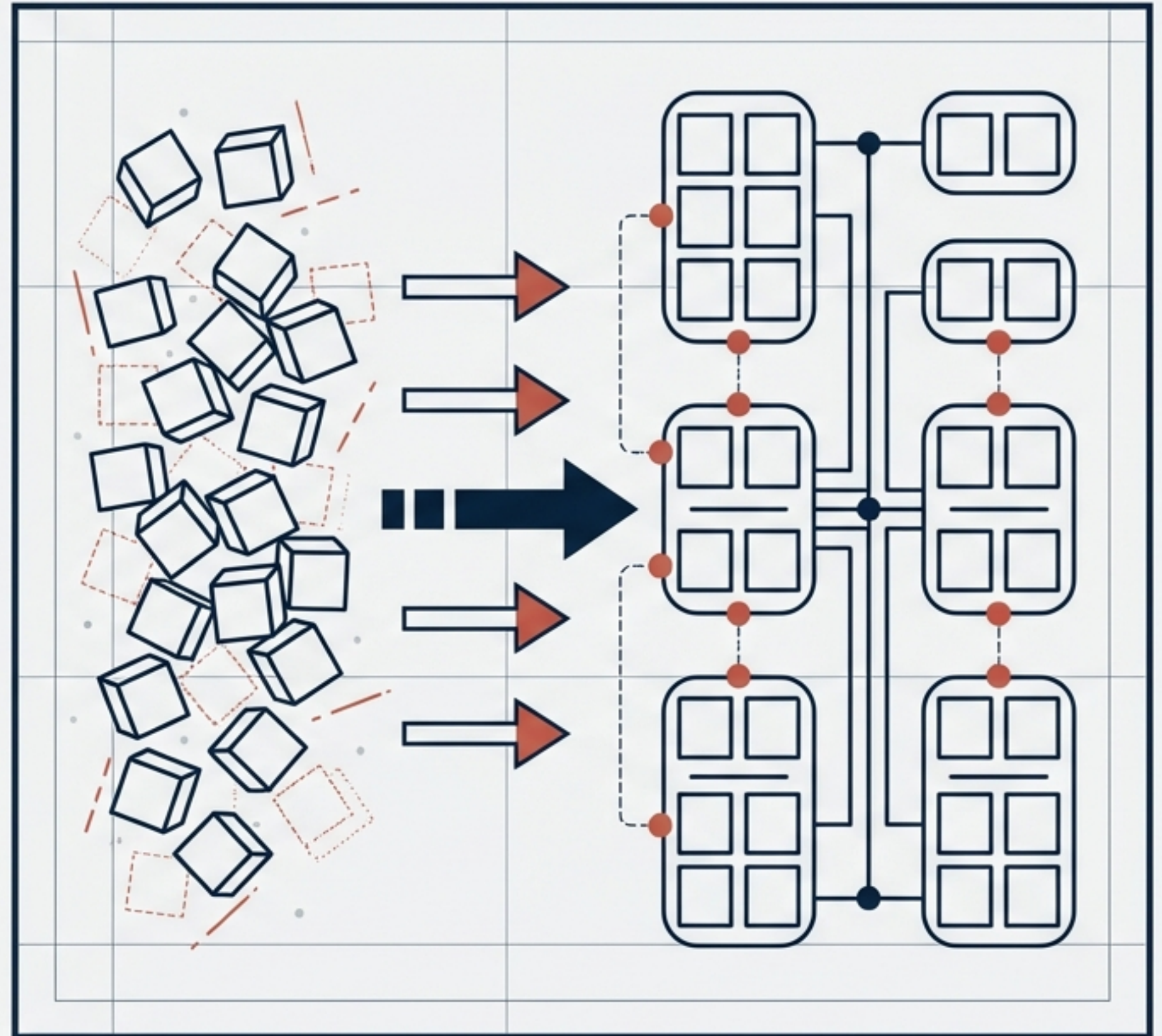
Demystifying Kubernetes Architecture

A top-down guide to container orchestration and cluster mechanics.

The Orchestrator of Distributed Software

Kubernetes is, at its core, container orchestration software. It is designed to manage distributed systems and keep them running efficiently.

Key Takeaway: The entire system works by wrapping and managing workloads inside fundamental units called "Pods".



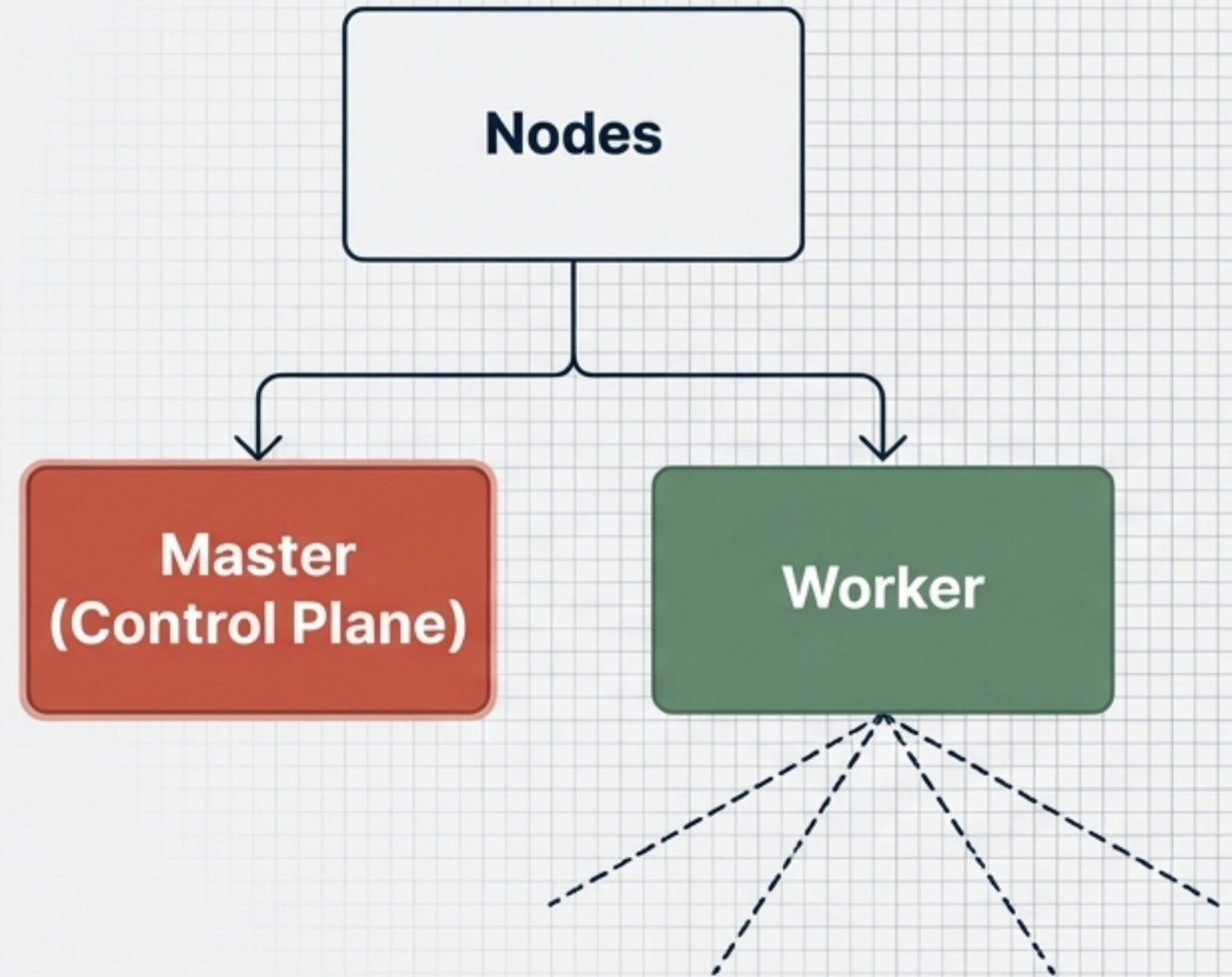
The Architectural Divide

A cluster is strictly divided into two distinct roles:

1. Master (The Control Plane)
2. Worker Nodes

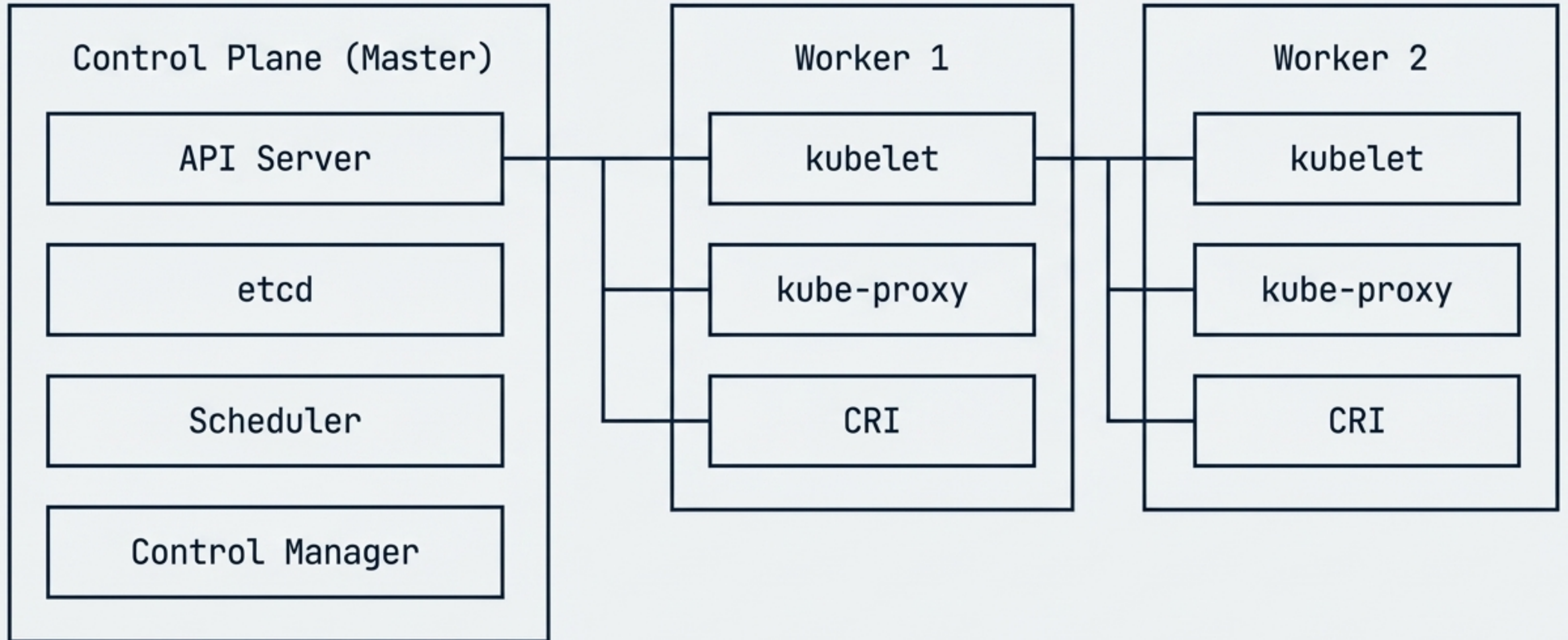
Deployment Realities:

- For learning and baseline mental models, we assume a 1 Master, 3 Worker architecture.
- In Development: 1 Master handles multiple Workers.
- In Production: The Master requires backup replicas to ensure high availability.



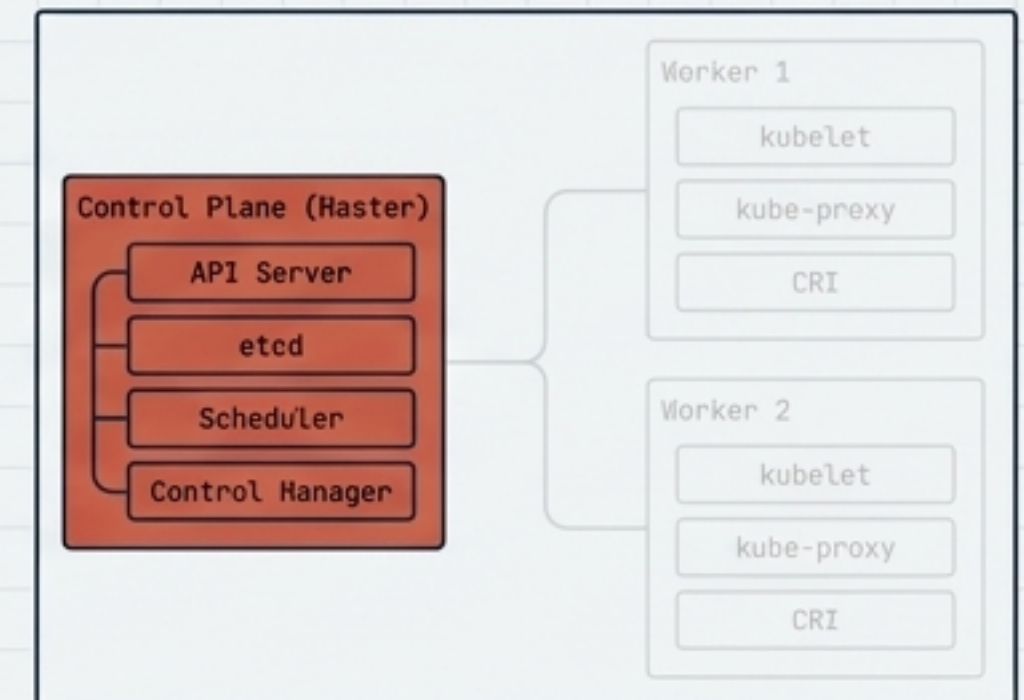
The Complete Cluster Map

A visual layout showing the Control Plane connecting to multiple Worker Nodes.



The Control Plane

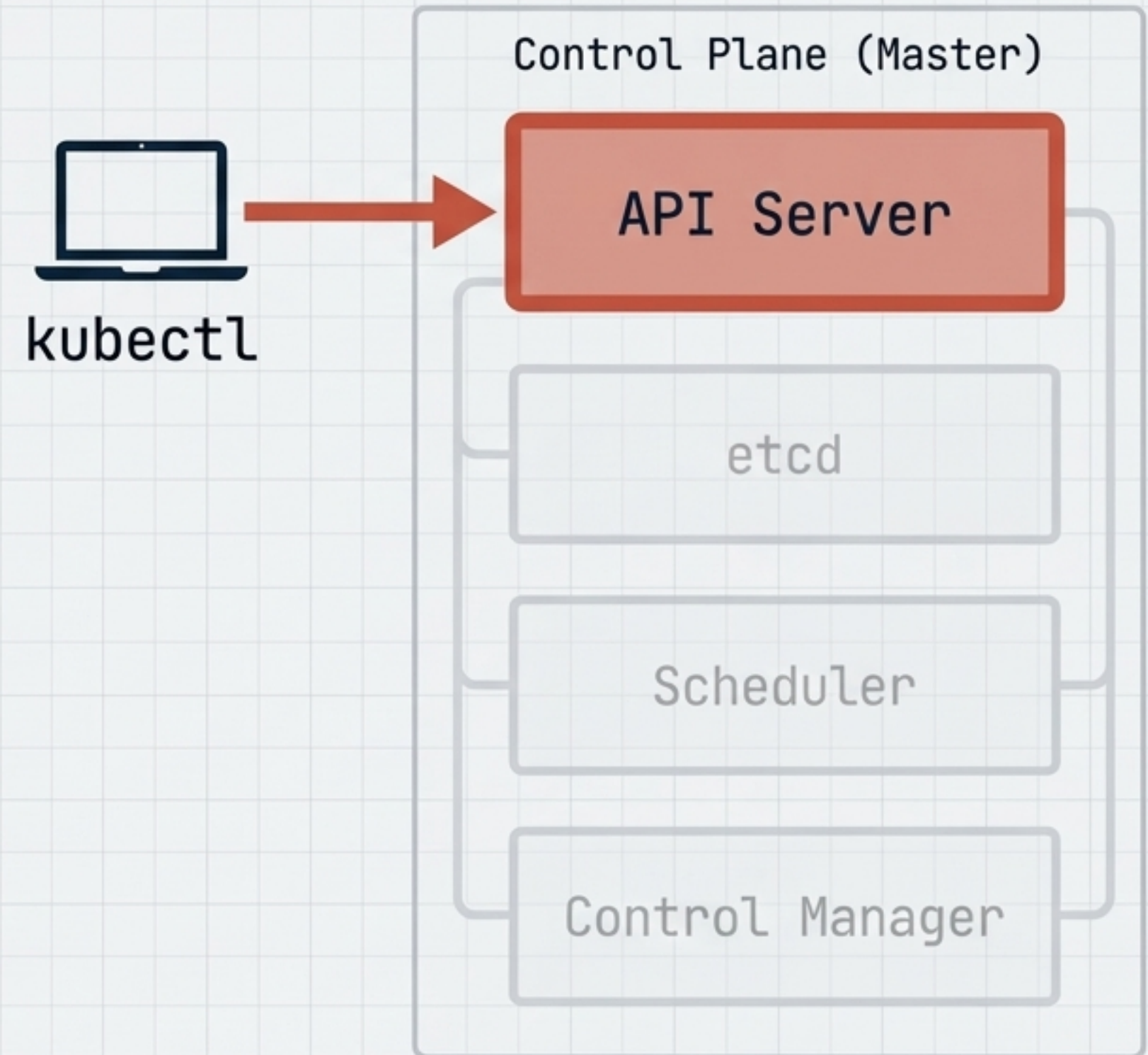
The Master node acts as the management hub of the entire cluster. It does not run user applications; instead, it dictates the global state of the cluster, makes scheduling decisions, and responds to cluster events.



API Server

The API Server is the exclusive entry point to the cluster.

- All communication—both internal and external—goes through it.
- It is responsible for handling all REST requests.
- When you execute a command using `kubectl`, you are interacting directly with the API Server.

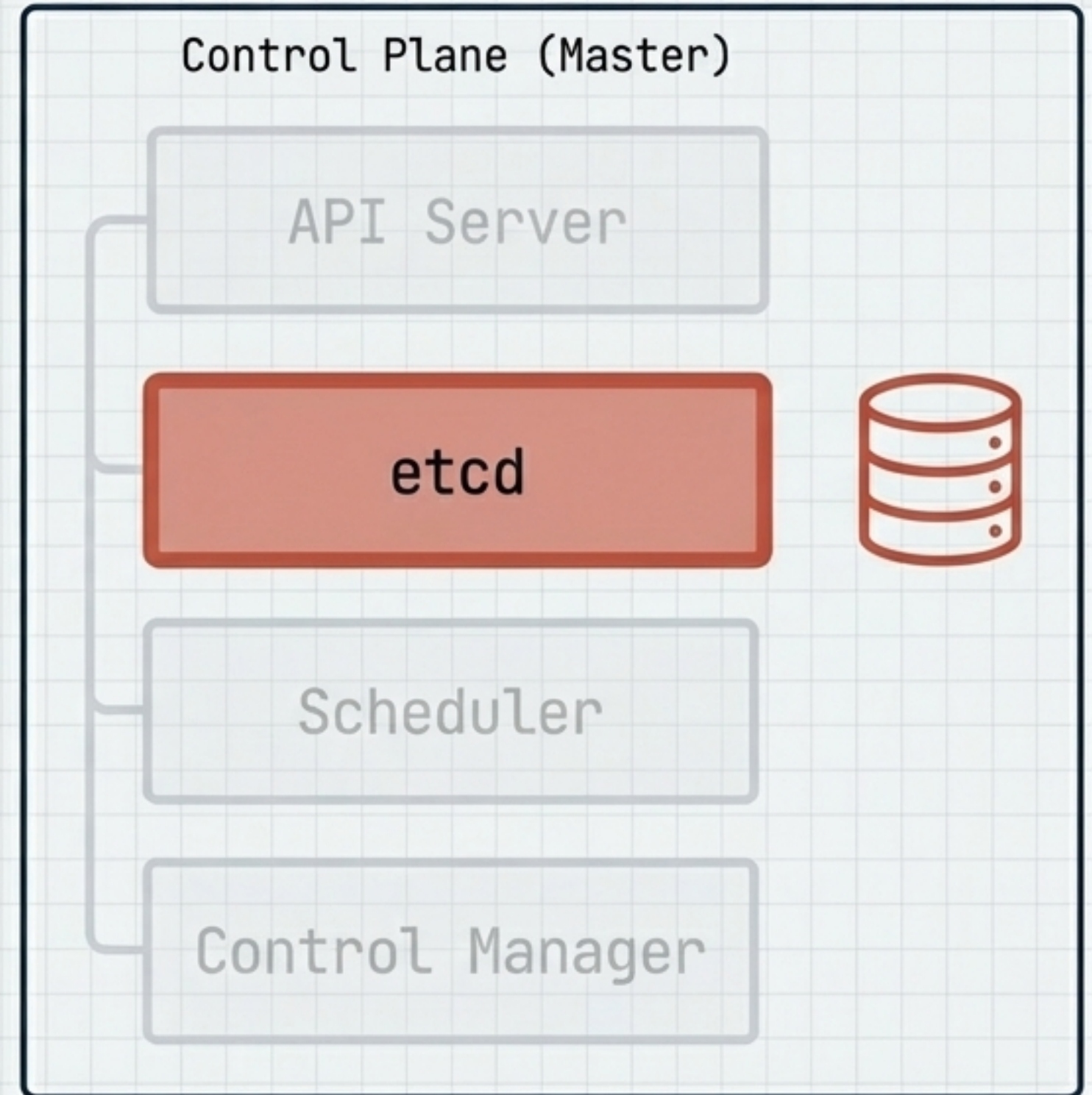


etcd

A highly available, distributed key-value store.

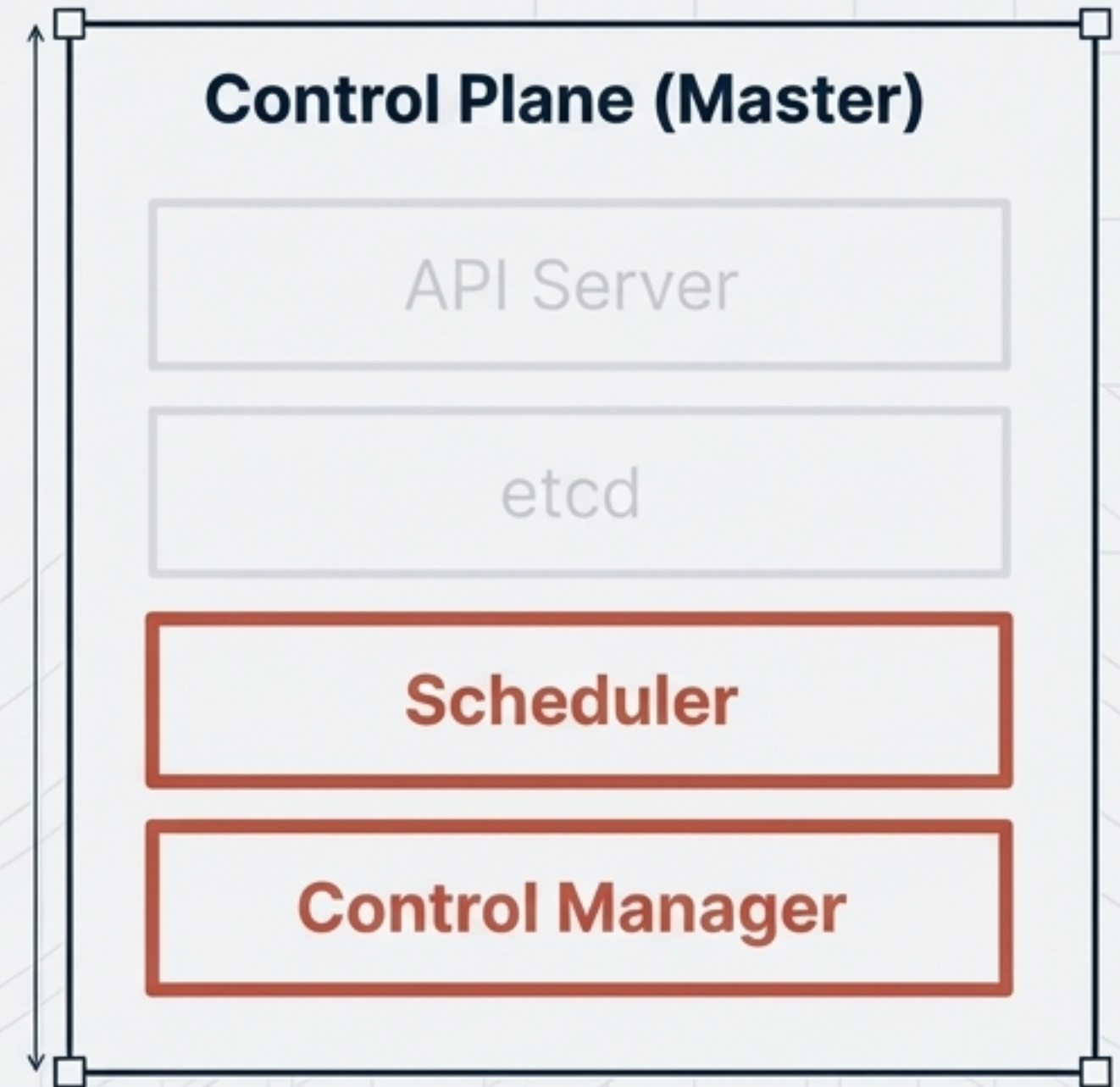
Role:

It serves as the cluster's ultimate source of truth, storing all cluster configuration data and current state information. If a component needs to know the state of the cluster, it queries etcd via the API Server.



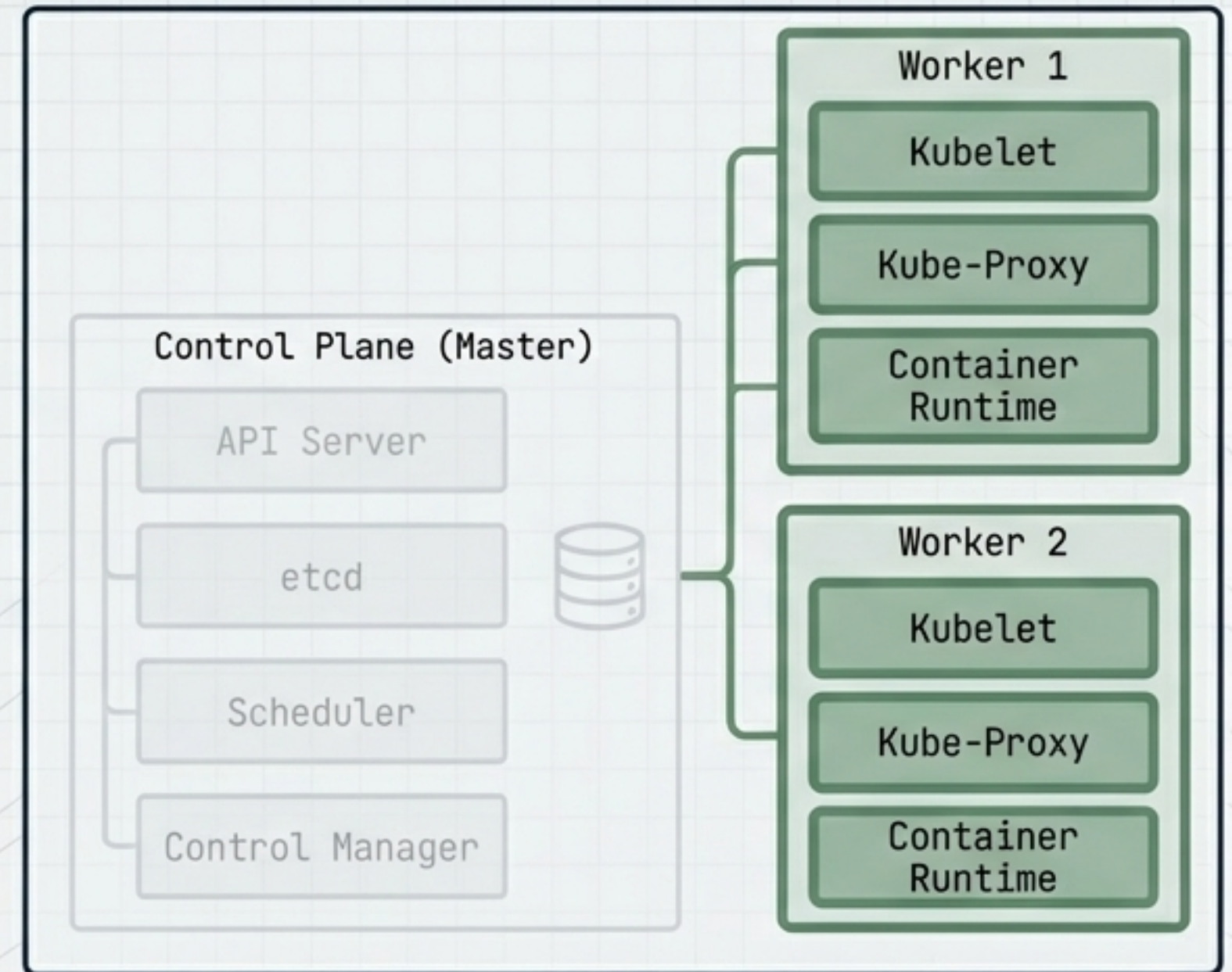
Scheduler and Control Manager

- **Scheduler:** The dispatcher. It assigns Pods to worker nodes, making the critical decisions about exactly where workloads will run based on resources.
- **Control Manager:** The enforcer. It ensures that the "desired state" of the cluster matches the "actual state". It actively handles replication tasks and monitors node health.



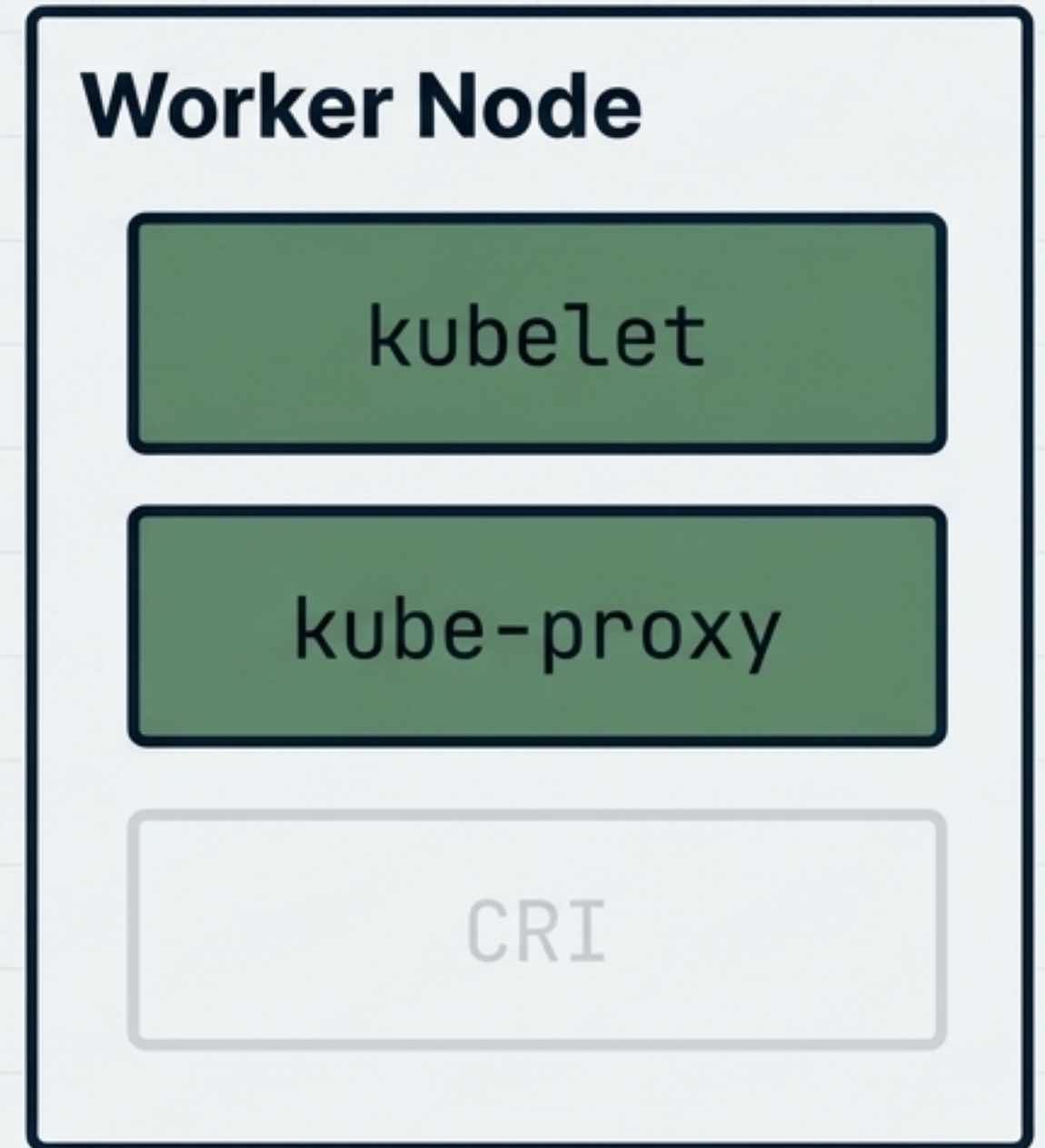
The Worker Nodes

If the Control Plane is the brain, the Worker Nodes are the brawn. These are the execution environments that provide the computing power. They receive instructions from the Control Plane and run the actual containerised applications.



kubelet and kube-proxy

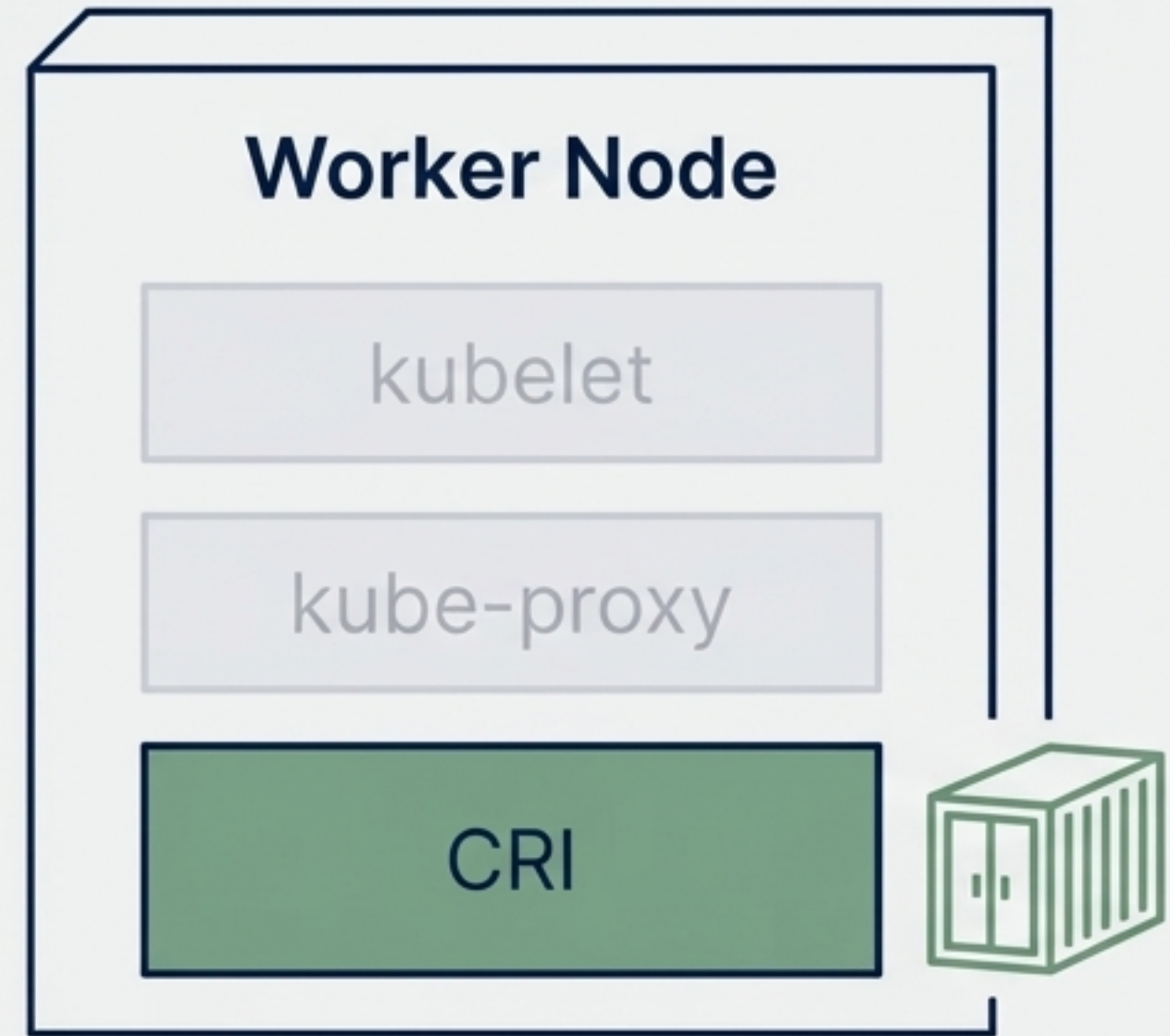
- **kubelet:** The primary agent running on each individual node. It communicates directly with the Control Plane and is responsible for ensuring that the assigned containers are actually running.
- **kube-proxy:** The local network manager. It manages network rules on the node and enables service communication across the cluster.



Container Runtime Interface (CRI)

Kubernetes does not run containers itself; it orchestrates them. The Container Runtime is the third-party engine installed on the worker node that contains containers. Examples: Docker and containerd.

Examples: Docker and containerd.



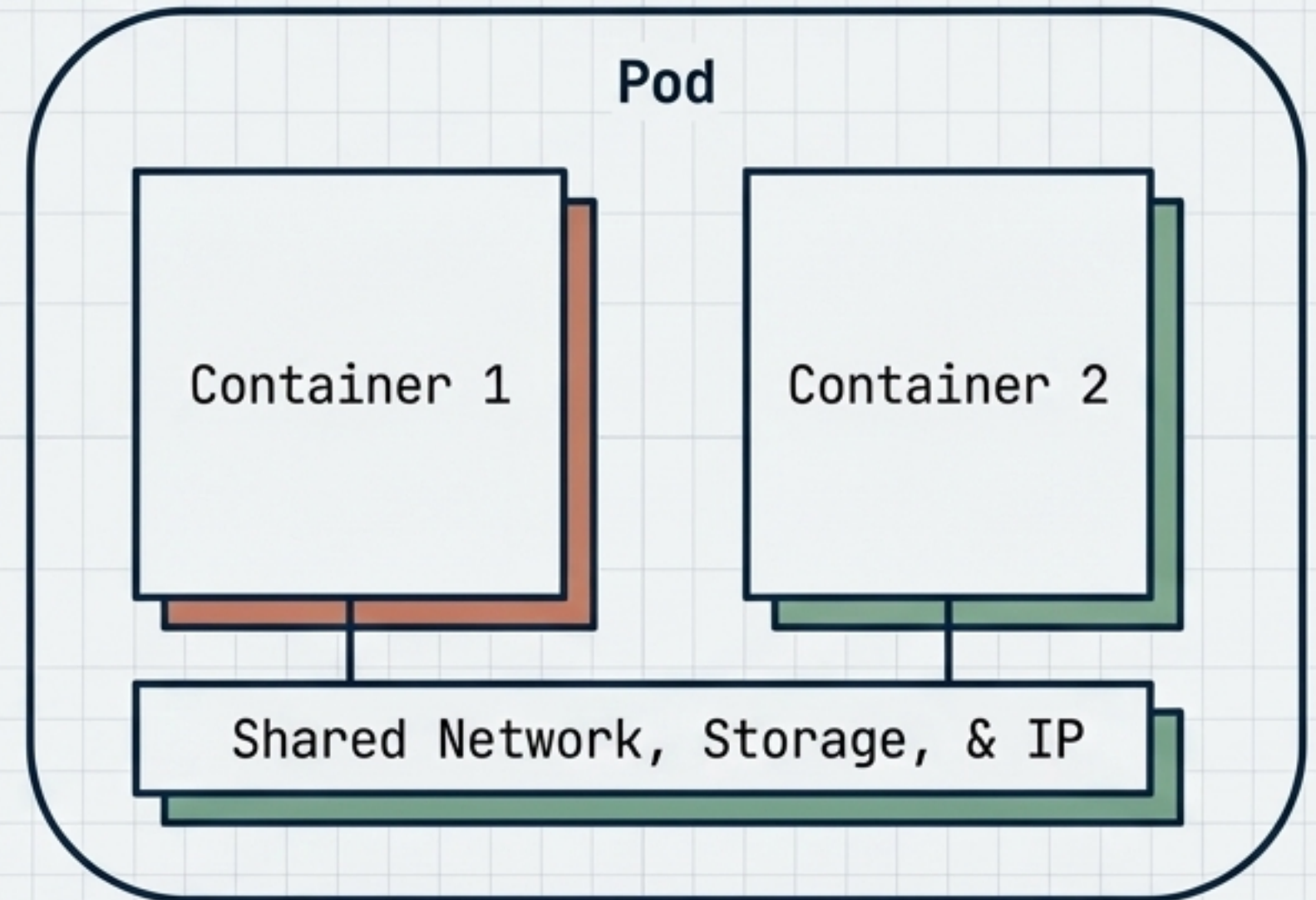
Core Application Concepts:

The Pod

The Pod is the smallest, most fundamental deployable unit in Kubernetes.

Key Characteristics:

- Can contain one or multiple containers.
- All containers within a Pod share the same **Network, Storage, and IP address**.
- Crucial reality: Pods are entirely ephemeral (temporary) by design. They are meant to be created and destroyed.

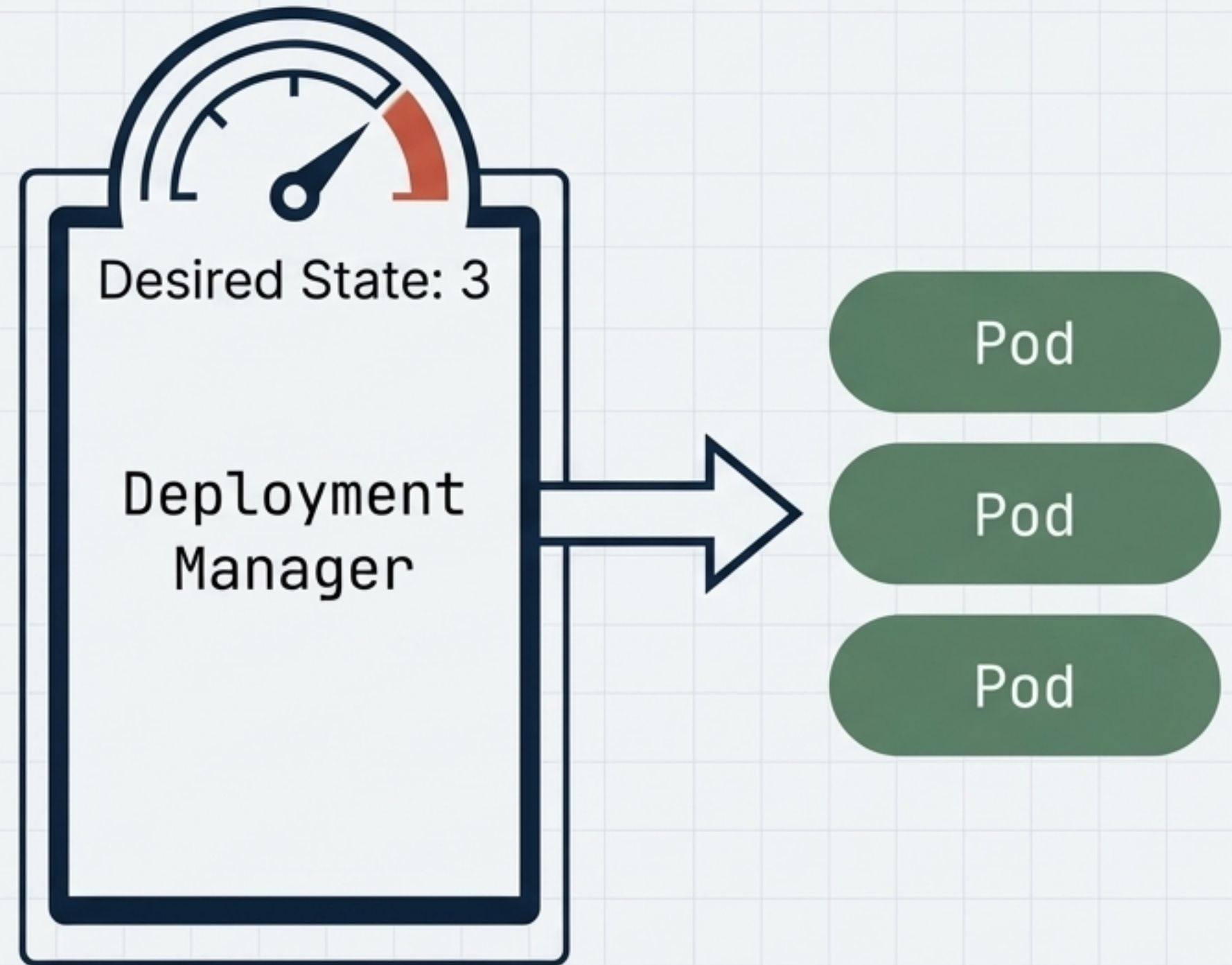


Deployments

Because Pods are ephemeral, they require management. A Deployment acts as a declarative manager for your Pods.

Capabilities:

- Manages Replicasets.
- Enables seamless rolling updates without downtime.
- Continuously ensures that the exact desired number of Pods are running at any given time.

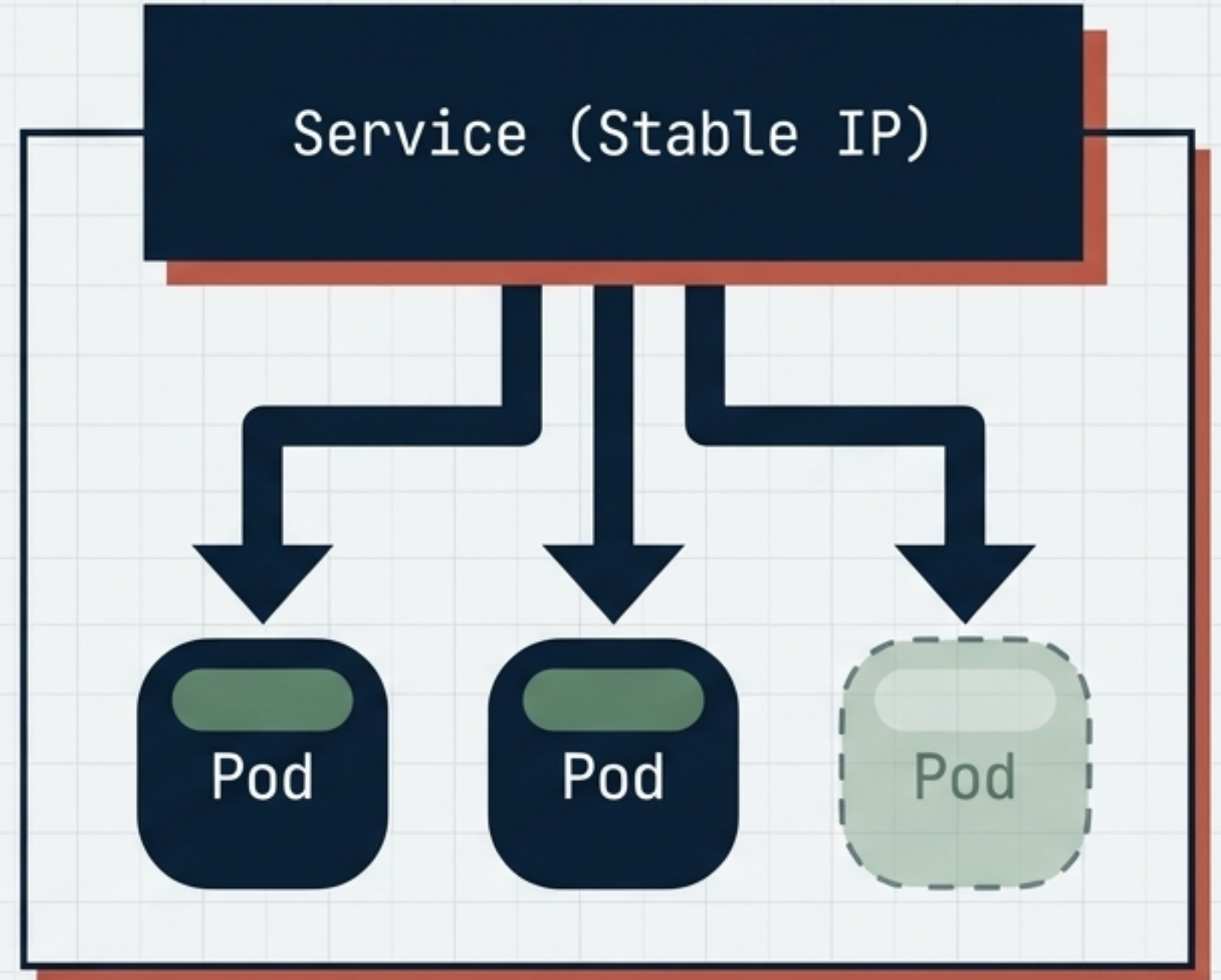


Services

If Pods are temporary, their IP addresses constantly change. A Service solves this by exposing Pods to the network and providing a stable, unchanging IP and DNS name.

Four Types of Services:

1. ClusterIP
2. NodePort
3. LoadBalancer
4. ExternalName



Inter-node Communication via Calico

Pods run on entirely different physical or virtual nodes, yet they must function as a single system. The Engine: Calico impressively handles the complex background routing and inter-node communication. It ensures that regardless of which node a Pod is running on, it remains reachable to the rest of the cluster.

